

**Universidad de Guadalajara**  
**Centro Universitario de Ciencias Exactas e Ingenierías**

División de tecnologías para la integración ciber-humana  
**Ingeniería en comunicaciones y electrónica**



Reporte del proyecto modular

**Implementación de un sistema de control automático de temperatura y PH  
para una línea de recubrimientos electrolíticos de Zinc-Niquel sobre  
aluminio.**

**Módulos por evaluar:**

Instrumentación y control  
Comunicaciones  
Circuitos digitales  
Circuitos analógicos

**Presenta(n):**

Salvador Castro Pérez  
Fernando Salvador Samayoa Martínez

**Asesor(es) del proyecto**

Omar Alejandro Gonzáles Meza, Alfredo Netzahualcoyotl Torres López  
Guadalajara Jalisco, 15 de mayo del 2026

## **1. Resumen**

El presente proyecto detalla el diseño e implementación de un sistema automatizado de instrumentación y control para optimizar el proceso de recubrimiento electrolítico de zinc-níquel sobre sustratos de aluminio. Debido a la susceptibilidad a la oxidación del aluminio, es crítico controlar estrictamente las variables fisicoquímicas del baño para garantizar la adherencia del recubrimiento y evitar acabados quebradizos. Para ello, se desarrolló una arquitectura de procesamiento distribuido Maestro-Esclavo. Un microcontrolador ESP32 gestiona una interfaz web asíncrona vía Wi-Fi, adquiere datos de pH y variables atmosféricas, calcula cuatro lazos de control PID de temperatura y gobierna una fuente de corriente continua/pulsada. Para aislar las interrupciones críticas y evitar latencias de red, un Arduino Nano maneja de forma independiente el control de fase AC mediante detección de cruce por cero y disparo de TRIACs. Esta separación garantiza alta precisión, proporcionando una infraestructura robusta que permite ejecutar Diseños de Experimentos (DoE) confiables para la industria galvanizadora.

## **2. Introducción**

El aluminio es un material ampliamente utilizado en la industria; sin embargo, posee una alta susceptibilidad a la oxidación espontánea. Esta reactividad frente al oxígeno genera capas de óxido que comprometen su integridad estructural a largo plazo y opacan rápidamente los acabados brillantes que se le aplican. Para mitigar estos efectos y prolongar la vida útil de las piezas, se emplean procesos electroquímicos de recubrimiento, destacando el uso secuencial de zinc y níquel. El recubrimiento de zinc actúa como un ánodo de sacrificio (protección catódica), mientras que la capa final de níquel proporciona resistencia a la corrosión y un acabado decorativo brillante.

A pesar de los beneficios teóricos de este tratamiento, existe una problemática actual documentada en la industria galvanizadora: al aplicar recubrimientos de níquel directamente o mediante procesos estándar sobre piezas de aluminio, el acabado resultante presenta mala adherencia, tornándose quebradizo y opaco. Para dar solución a esta deficiencia, una investigación en curso del área de Ingeniería Química propone la validación de una formulación específica de zincado en medio ácido como capa de adherencia previa al niquelado.

Para que dicha investigación química pueda encontrar las condiciones óptimas de calidad en el recubrimiento, se requiere la ejecución de un Diseño de Experimentos (DoE) riguroso, el cual evaluará la influencia de múltiples variables fisicoquímicas. Es precisamente en esta etapa donde surge la necesidad crítica que aborda este proyecto modular. La estabilización del proceso electrolítico y la obtención de un modelo predictivo confiable son imposibles sin un control estricto de las variables de operación.

Por lo tanto, el presente trabajo se centra en el diseño e implementación de un sistema de experimentación automatizado capaz de intervenir en las etapas de limpieza, decapado, zincado y niquelado. El sistema de instrumentación y control propuesto proporcionará la capacidad de medir y regular la temperatura de los contenedores, monitorear el pH de las soluciones y suministrar energía a través de una fuente capaz de controlar con precisión la densidad de corriente, permitiendo además la alternancia entre modos de corriente continua y corriente pulsada. Esta infraestructura tecnológica es el pilar que permitirá evaluar las condiciones experimentales del baño electrolítico y garantizar la calidad final de los recubrimientos de Zn-Ni sobre los sustratos de aluminio.

### 3. Objetivos

Objetivo general

Diseñar e implementar un sistema de instrumentación, control y registro de datos con parámetros (*setpoints*) modificables para las variables críticas (pH, temperatura y corriente) en el proceso de recubrimientos electrolíticos.

Objetivos Particulares.

Implementar un sistema de control de lazo cerrado para la regulación térmica independiente en cada uno de los recipientes del proceso.

Desarrollar un sistema de adquisición de datos para el monitoreo continuo del pH, la temperatura y la corriente eléctrica aplicada.

Diseñar y construir una fuente de corriente directa con capacidad de modulación de pulsos, permitiendo la configuración precisa de su amplitud, ciclo de trabajo y frecuencia.

Crear una interfaz de comunicación inalámbrica que permita el almacenamiento histórico de datos y la modificación remota de los parámetros experimentales.

Integrar todas las etapas de acondicionamiento, control y potencia dentro de la arquitectura de un sistema embebido.

### 4. Fundamentación teórica

#### 4.1 Modelado Térmico

El comportamiento térmico general de un fluido continuo con generación de calor está gobernado por la ecuación diferencial parcial de energía en su forma vectorial:

$$\rho c_p \left( \frac{\partial T}{\partial t} + \mathbf{v} \cdot \nabla T \right) = k \nabla^2 T + \dot{q}_{vol}$$

Donde

$\frac{\partial T}{\partial t}$  es la tasa de cambio de temperatura con el tiempo

$\rho c_p$  es la capacidad calorífica volumétrica

$v \cdot \nabla T$  es el Termino de advección

$k\nabla^2 T$  es el termino de difusión

$\dot{q}_{vol}$  es la generación de calor interno

Esta ecuación diferencial parcial es la representación general de un sistema térmico, pero cuando nosotros introducimos en el sistema una convección forzada se puede modelar el sistema como uno de parámetros concentrados donde el calor es igual en todos los puntos del sistema esto permite pasar de una ecuación dependiente del espacio y el tiempo a una ecuación diferencial de primer orden representada por

$$mc_p \frac{dT}{dt} = \dot{Q} - hA_s(T(t) - T_\infty)$$

donde

$$mc_p \frac{dT}{dt} = \text{Energia Almacenada en el Sistema}$$

$$\dot{Q} = \text{Energia proporcionada al sistema}$$

$$hA_s(T(t) - T_\infty) = \text{Perdidas del sistema}$$

Esta ecuación puede ser aplicada a todos los sistemas térmicos cuando se conoce la Masa ( $m$ ), Calor Especifico( $c_p$ ), Cambio de Temperatura con respecto al tiempo ( $\frac{dT}{dt}$ ), la energía proporcionada al sistema ( $\dot{Q}$ ) el coeficiente de convección térmica( $h$ ), área de superficie ( $A_s$ ) temperatura actual ( $T(t)$ ) y temperatura ambiente( $T_\infty$ ).

## 4.2 PID

Para la regulación térmica, se utiliza el algoritmo PID, el cual calcula la acción de control basándose en el error entre el setpoint deseado y el valor medido. La ecuación teórica en el dominio del tiempo es:

$$u(t) = k_p e(t) + k_i \int_0^t e(t) dt + K_d \frac{de(t)}{dt}$$

Donde:

$k_p$  Produce una salida proporcional al error actual.

$k_i$  Elimina el error en estado estacionario acumulando el error pasado.

$K_d$  Predice el comportamiento futuro del error basándose en su tasa de cambio, mejorando la estabilidad.

Este será el controlador implementado al sistema.

#### 4.3 Fuente de corriente controlada por voltaje

Una fuente de corriente controlada por voltaje (VCCS, por sus siglas en inglés) es un dispositivo electrónico diseñado para ajustar y mantener una corriente de salida proporcional a un voltaje de referencia aplicado. La principal característica de esta topología es que el parámetro fijado y controlado por el sistema es la corriente, permitiendo que el voltaje de salida varíe dinámicamente en función de la carga conectada.

Esta propiedad es un factor clave en los procesos de recubrimiento. Durante la electrodeposición, la impedancia del baño electrolítico es sumamente baja y se modifica constantemente a medida que transcurre el proceso químico. Si se utilizara una fuente de voltaje convencional, estas fluctuaciones resistivas generarían variaciones severas en la corriente aplicada, lo que afectaría drásticamente la calidad, uniformidad y adherencia del recubrimiento final.

Matemáticamente, el comportamiento de este sistema se describe mediante una relación lineal general:

$$I_{out} = G_m V_{ref}$$

Donde

$I_{out}$  es la corriente de salida

$G_m$  es la dinámica de la fuente o transconductancia

$V_{ref}$  el voltaje de referencia o control

#### 4.4 Wi-Fi

El Wi-Fi (basado en el estándar IEEE 802.11) es una tecnología de comunicación inalámbrica de alta velocidad que permite la creación de redes de área local (WLAN). La transferencia de datos mediante esta tecnología opera bajo la arquitectura TCP/IP utilizando un modelo Cliente-Servidor. Desde una perspectiva teórica, un sistema embebido puede fungir como un servidor web que atiende peticiones HTTP entrantes a través de esta red. Al recibir una solicitud, el servidor transmite archivos estructurados (HTML/CSS) que el navegador de un dispositivo cliente interpreta para renderizar una interfaz gráfica. Esto permite el despliegue de paneles de control interactivos de forma remota, prescindiendo por completo de conexiones físicas.

## 5. Metodología y materiales

El diseño del sistema de instrumentación se derivó directamente de los parámetros operativos establecidos en el protocolo de investigación para el recubrimiento de Zinc-Níquel sobre Aluminio 6061-T6.

El diseño del sistema de calentamiento se basó en los requerimientos específicos de cada etapa del proceso de recubrimiento. Se definieron temperaturas de consigna de 90 °C para las soluciones de decapado y limpieza (volumen de 1 L), 40 °C para el baño de niquelado (1 L) y 40 °C para la solución ácida de zincado (267 mL).

Para determinar la capacidad calorífica necesaria, se utilizó la ecuación de transferencia de energía térmica:

$$P = \frac{m c_p \Delta T}{t}$$

donde

P es la potencia necesaria para calentar el líquido

m es la masa por calentar

$\Delta T$  la diferencia de temperatura ambiente y objetivo

t el tiempo necesario

Tras la sustitución de los valores objetivos (usando un tiempo de referencia de 25 minutos), se observó que eran necesarios aproximadamente 225 W para las soluciones de decapado, 50 W para el niquelado y 13 W para el zincado en celda. Basándose en estos resultados y en la disponibilidad comercial de componentes, se implementaron los siguientes actuadores:

450 W para limpieza y decapado: Tuvimos que elegir las sobradas debido a que se necesita hacer un encapsulado para evitar la corrosión, ya que las sustancias de decapado y limpieza son muy básicas.

Para las etapas restantes: Elegimos resistencias que ya existían en el proyecto con anterioridad: una de 18 W para el zincado y una de 36 W para el niquelado. Si bien es de una potencia menor, esta última tiene un tiempo extra para llegar a la temperatura deseada, además de que en esta etapa la placa durará 30 minutos según el DoE.

Una vez elegidas las resistencias, se buscó la forma de controlarlas. Si bien tenemos en mente un PID, este es solo el algoritmo de control y se necesita un actuador. Al ser nuestras resistencias de una potencia alta, son de corriente alterna, por lo que hubo la necesidad de implementar un control por TRIACs. Estos los conseguimos en módulos prefabricados como atenuadores de luz; funcionan perfecto, ya que para el TRIAC no existe diferencia entre una resistencia calefactora y un foco de filamento de tungsteno, dado que ambas son cargas resistivas.

Nos inclinamos por el módulo MDAC4C de JELELECTRONICA, el cual cuenta con 4 módulos de control manejados por interrupciones en cruce por cero. El mismo

módulo cuenta con aislamiento galvánico por medio del optoacoplador MOC3063 y TRIACs BTA24-800.

Una vez realizada la elección de los elementos de control, procedimos a elegir los sensores para la adquisición de datos. El experimento ya tenía material comprado (como ocurrió con las resistencias), así que decidimos usar los termopares ya elegidos, los cuales son sumamente robustos para entornos de alta temperatura y corrosión. Usamos el MAX6675, que consta del termopar tipo K, su amplificador, compensador de junta fría, ADC y transceptor para conectarlos por protocolo SPI. Para los sensores de pH se eligieron electrodos E201C-BNC de plata/cloruro de plata con conector BNC y el módulo PH-4502C. Este cuenta con un amplificador y *trimpots* de ajuste para dejarlo en escala lineal; es un sensor completamente analógico con una alta demanda de corriente y requiere 5 V estables de suministro para su funcionamiento. Para medir las condiciones atmosféricas, elegimos un sensor todo en uno: una placa que contiene el AHT20 + BMP280 y se conecta por I2C.

Como cerebro de nuestra operación elegimos el ESP32-S3, que es un MCU de alta velocidad con capacidades nativas de conectividad Wi-Fi, lo que nos permite el desarrollo de una interfaz web para no tener que usar botones. Esto mismo hace que usar interrupciones sea sumamente complejo, con el riesgo de cortar la comunicación Wi-Fi. Por eso mismo, optamos por un sistema maestro-esclavo conectado por UART, donde el ESP32-S3 recibe las señales de temperatura y envía las señales de control a un Arduino Nano, el cual se encarga únicamente del control de cruce por cero por medio de interrupciones de los TRIACs, sin el riesgo de bloquear la señal Wi-Fi ni la interfaz.

Finalmente, como tenemos señales de todo tipo (digitales, analógicas, AC y DC), decidimos usar DAC y ADC externos al MCU para una mejor estabilidad. El ADC se encarga de convertir a I2C los sensores analógicos de 5 V (como el de pH) a un formato manejable por el ESP32-S3, sin necesidad de divisores de tensión que introduzcan ruido. El DAC, por su parte, generará la referencia para nuestra fuente de corriente; así podemos generar de forma digital tanto la señal continua como la señal pulsada necesaria.

Para el suministro de corriente decidimos implementar un sumidero de corriente (o fuente de corriente de lado bajo) por medio de transistores MOSFET de canal N IRLZ-44, los cuales son dispositivos de potencia, y como elemento de control, el amplificador operacional LM358 en forma de amplificador de error, midiendo la corriente del transistor por medio de una resistencia *shunt* de 1 Ohm a 10 W.

Finalmente, para alimentar nuestro circuito decidimos usar una fuente conmutada de 12 V a 10 A, que entrega corriente de sobra, además de reguladores lineales.

LM7805 y LM317 para los voltajes de 5 y 3.3 necesarios se usó un conversor Buck para bajar el voltaje a 6 V para no tener un calentamiento excesivo en los reguladores lineales que filtran el voltaje con ruido armónico de la fuente conmutada.

## 6. Desarrollo del prototipo

Una vez elegidos todos los componentes primeramente llevamos el modelo térmico particular a el caso específico donde sustituimos la potencia aplicada al tanque con la potencia producida por el TRIAC de esta forma

$$mc_p \frac{dT}{dt} = \left( \left( \frac{V_s^2}{R} * \frac{1}{2\pi} \right) \left( \pi - \alpha + \frac{\sin(2\alpha)}{2} \right) \right) - hA_s(T(t) - T_\infty)$$

Así tenemos una ecuación con claras variables de control y a controlar Las cuales son temperatura(variable a controlar) y ángulo de disparo(variable controladora). Nuestro objetivo es llevar nuestra ecuación a la forma

$$G(s) = \frac{\Delta T(s)}{Q(s)} = \frac{K}{\tau s + 1} e^{st}$$

Donde

$$K = \frac{1}{h * A_s}$$

$$\tau = \frac{m * C_p}{h * A_s}$$

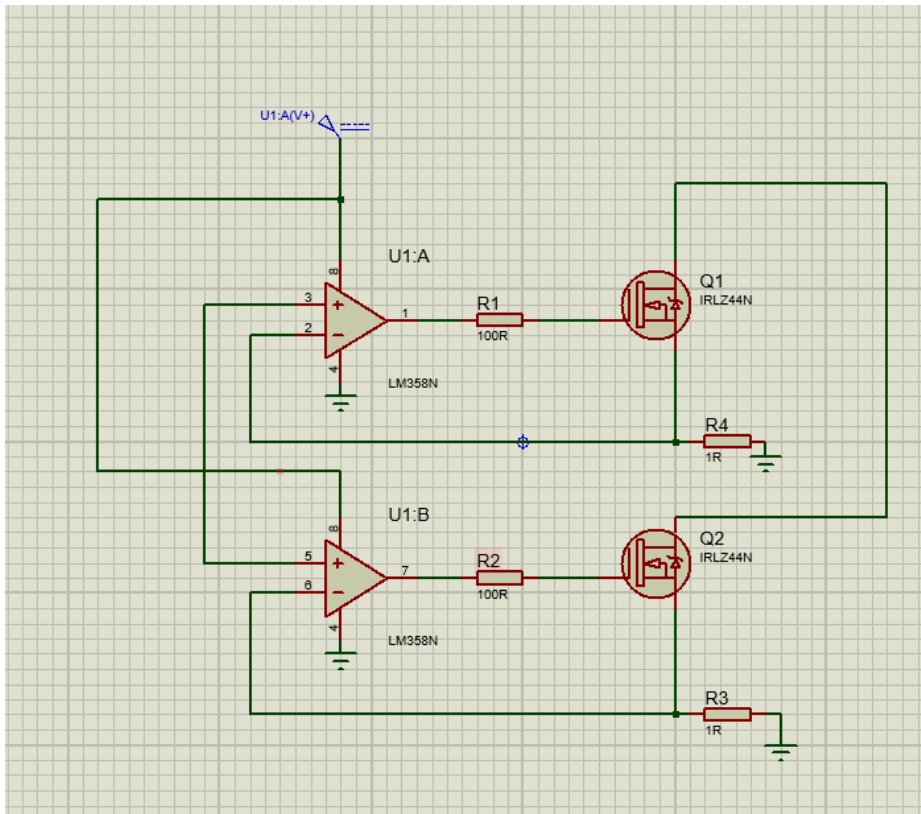
$$t = \text{retardo del sensor}$$

Se implementa un retardo ya que las resistencias están encapsuladas y el sensor no esta al lado de la resistencia.

Con este modelo es posible generar un código de MATLAB donde al correrlo nos dará diferentes ganancias de Kp, Ki, Kd.

Al correr el código fue evidente que el sistema no necesitaba un derivador gracias a la gran inercia térmica de los líquidos al calentarse y su dinámica lenta el Kd calculado por los algoritmos de MATLAB fueron ganancias negativas las cuales indican que no son necesarias

Después de esto se desarrollo el sumidero de corriente este tiene la forma mostrada en el siguiente esquemático



*Imagen 1 Esquemático sumidero de corriente*

Este sumidero de corriente es un circuito sumamente estable para cargas de baja potencia como las tinas de electroplating el circuito funciona con el LM358 configurado como amplificador de error este se encarga de sentir el voltaje aplicado en la resistencia shunt de 1ohm para amplificar el voltaje de conducción del IRLZ44 + el voltaje de referencia dando una forma de controlar el paso de corriente por medio de voltaje el uso de 2 ramas de control combinadas en el Drain nos genera una ganancia de 2 siemens donde por cada volt aplicado se generan 2 amperes, esto ayuda a la gestión térmica de los MOSFETs repartiendo 750mA por rama con un voltaje de 5V cada MOSFET ya que experimentalmente con una fuente de laboratorio comprobamos que las tinas necesitan un voltaje de 5.56 V para que circulen 1.5 Amperes esto quiere decir que existen 6.54 A libres menos el 1.5 V aplicados al resistor shunt, el mosfet tiene que disipar 5 volts y 750mA lo que da una disipación de alrededor de 4W esto es manejable con disipadores sin ventilación forzada y mantienen el MOSFET en rangos de operación seguros

El siguiente análisis matemático describe el funcionamiento del circuito

Empezamos con

$$V_{out} = A_o(V^+ - V^-)$$

$$V_{out} = A_o(V_{ref} - (I_d * R_s))$$

El cuál es la ecuación de salida de un amplificador no inversor con realimentación después realizamos un análisis de mallas donde observamos que

$$V_{out} = V_{gs} + (I_{ds} * R_s)$$

Igualando ambas ecuaciones podemos

$$I_d = \frac{V_{ref} - \frac{V_{gs}}{A_o}}{R_s + \frac{R_s}{A_o}}$$

Tomando en cuenta que  $A_o$  es prácticamente infinito entonces  $\frac{V_{gs}}{A_o}$  y  $\frac{R_s}{A_o}$  es igual a 0

$$I_d = \frac{V_{ref}}{R_s}$$

Y como  $R_s = 1$  tenemos nuestra relación

$$I_d = V_{ref}$$

Lo que nos da una fuente de corriente controlada por voltaje con una relación lineal 1-1 y el uso de dos ramas nos deja con una ganancia de 2

Para la siguiente parte lo necesario es programar nuestro ESP-32 S3 y Arduino nano para que usen puedan cumplir sus funciones

El ESP32-S3 actúa como la unidad de toma de decisiones. Utilizando las ganancias  $K_p$  y  $K_i$  sintonizadas previamente en MATLAB, el microcontrolador ejecuta en cada ciclo de muestreo la ecuación diferencial del controlador: Cálculo de la Salida: El software compara la temperatura real contra el setpoint. El error resultante se multiplica por las ganancias para obtener una Potencia Nominal expresada en un porcentaje de 0 a 100%.

Transmisión Serial: Este valor porcentual se envía vía UART al Arduino Nano. La separación de tareas permite que el ESP32 gestione el stack de comunicaciones Wi-Fi sin que las operaciones matemáticas pesadas interfieran con la velocidad de respuesta del control de potencia

Sincronización por Interrupción y Tiempo de Retardo Una vez que el Arduino Nano recibe el valor de potencia, entra en juego la librería AcPhaseControl. El control de fase no puede ser aleatorio; debe estar perfectamente sincronizado con la red eléctrica de 60 Hz: La Interrupción de Cruce por Cero: El hardware detecta el momento exacto en que la onda senoidal cruza los 0V y dispara una interrupción externa en el Nano.

Cálculo del Retardo

En el momento del cruce por cero, el Nano inicia un temporizador. La librería traduce el porcentaje enviado por el ESP32 en un tiempo de espera en microsegundos. Si se requiere el 100% de potencia, el retardo es de 0  $\mu$ s (se deja pasar toda la onda). Si se requiere el 50% de potencia, el retardo es de aproximadamente 4166  $\mu$ s (la mitad del semiciclo de 8.33ms). Si se requiere 0% de potencia, el Nano simplemente no genera ningún pulso

**Voltaje en Gate y Activación del TRIAC** Finalmente, tras cumplirse el tiempo de retardo calculado, el Arduino Nano envía un pulso digital de 5V al optoacoplador MOC3063: **Aislamiento y Gate:** El optoacoplador permite que la señal de baja tensión del microcontrolador active la compuerta o Gate del TRIAC BTA24 mediante un flujo de fotones, aislando eléctricamente el circuito lógico de los 120V AC

**Conducción:** El voltaje aplicado en el Gate permite que el TRIAC pase al estado de conducción. Una propiedad fundamental del TRIAC es que, una vez activado por este pulso de voltaje, permanecerá "encendido" (dejando pasar corriente a la resistencia) por el resto del semiciclo, incluso si el voltaje en el Gate desaparece. El dispositivo se apagará automáticamente al final del semiciclo cuando la corriente caiga nuevamente a cero, repitiendo el ciclo 120 veces por segundo (dos veces por cada ciclo de 60 Hz).

### Integración de Periféricos y Abstracción por Librerías

La estabilidad del prototipo se basa en la precisión de la adquisición y generación de señales. Para esto, se utilizaron módulos externos gestionados mediante librerías de código abierto que simplifican protocolos de comunicación complejos.

#### Digitalización de Sensores (I2C y SPI)

El uso de las librerías de Adafruit y MAX6675 permitió que el ESP32-S3 interactuara con el hardware mediante funciones de alto nivel, abstrayendo la manipulación de bits en los protocolos:

**Lectura de pH (ADS1115):** Mediante la librería Adafruit\_ADS1X15.h, se configuró el ADC externo para operar en modo de ganancia ajustable. Esto permitió leer los electrodos de pH con una resolución de 16 bits, superando la limitación del ADC interno del MCU y filtrando el ruido eléctrico de la fuente.

**Monitoreo Térmico (MAX6675):** La librería max6675.h gestiona la comunicación SPI con los termopares. Su función principal es leer la trama de datos digitalizados, aplicar la compensación de unión fría internamente y entregar al programa una variable tipo float lista para ser usada en el error del PID.

Variables Atmosféricas (AHT20+BMP280): A través de Wire.h, se habilitó un bus I2C compartido. Las librerías correspondientes permiten obtener humedad y presión con una sola línea de comando (getEvent), lo que facilita el registro de las condiciones ambientales que afectan la evaporación del baño electrolítico.

#### Generación de Referencia de Corriente (MCP4725)

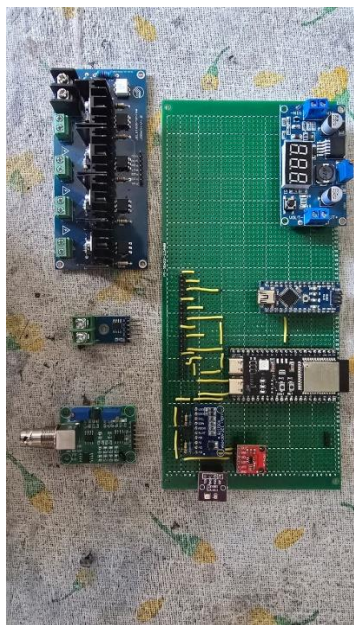
Para el control del sumidero de corriente, se utilizó el DAC de 12 bits MCP4725. La librería Adafruit\_MCP4725.h es crítica en este punto porque permite enviar valores digitales (0-4095) que el módulo convierte instantáneamente en voltajes analógicos precisos.

Esta abstracción fue fundamental para implementar la corriente pulsada de 10 Hz: el código simplemente envía un valor de amplitud durante 20 ms y un valor de cero durante 80 ms. La librería se encarga de que esta transición en el bus I2C sea lo suficientemente veloz para mantener la forma de onda cuadrada requerida.

El monitoreo de acidez, crítico para la deposición del zinc-níquel, se implementó mediante el módulo PH-4502C acoplado a un electrodo analógico de Plata/Cloruro de Plata (E201-BNC). Este sensor entrega una señal en milivoltios que es proporcional a la concentración de iones en la solución, la cual es preamplificada y digitalizada por el ADC externo (ADS1115) de 16 bits para filtrar el ruido eléctrico y asegurar una lectura de alta precisión.

Dada la deriva natural de este tipo de electrodos por el desgaste de su electrolito interno, el sistema utiliza un protocolo de calibración lineal físico apoyado en tres soluciones buffer (pH 4.0, 7.0 y 10.0). El microcontrolador ESP32-S3 almacena los voltajes de referencia de estos puntos para calcular la pendiente y el offset de la recta ideal ( $y=x+b$ ), aplicando esta compensación matemática en tiempo real a cada medición.

Para garantizar la estabilidad del prototipo a largo plazo frente a la contaminación química, el firmware integra un modo de calibración accesible directamente desde la interfaz web asíncrona. Esto permite al usuario ajustar los parámetros de la ecuación en cualquier momento, asegurando la fiabilidad metrológica de la línea de recubrimientos sin necesidad de reprogramar el equipo.

[illegible]

## 7. Resultados y discusión

## 7. Resultados y discusión

El primer paso para validar el proyecto fue graficar el controlador en MATLAB para ver el seguimiento, esfuerzo y la evolución del error el cual arrojo lo siguiente

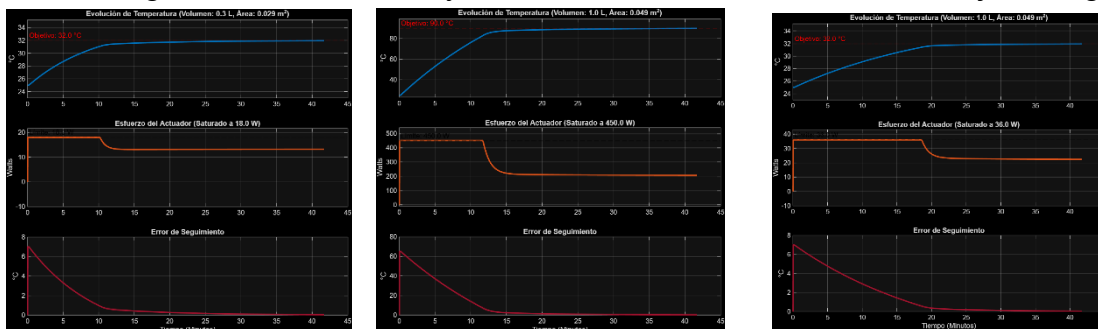


Imagen 4 Graficas de control de Zincado

Imagen 5 Graficas de control de niquelado

Imagen 6 Graficas de control del niquelado

Como podemos observar en las gráficas ya que el diseño de experimentos da rangos de temperaturas de 10 grados usamos una temperatura de setpoint para calcularlo de del promedio de los valores posibles así funciona correctamente hacia el límite inferior como superior del rango de temperaturas.

Después se decidió validar la interfaz para saber si era legible y arrojaba datos reales además de no bloquearse por interrupciones lo siguiente fueron los resultados de cada sección.

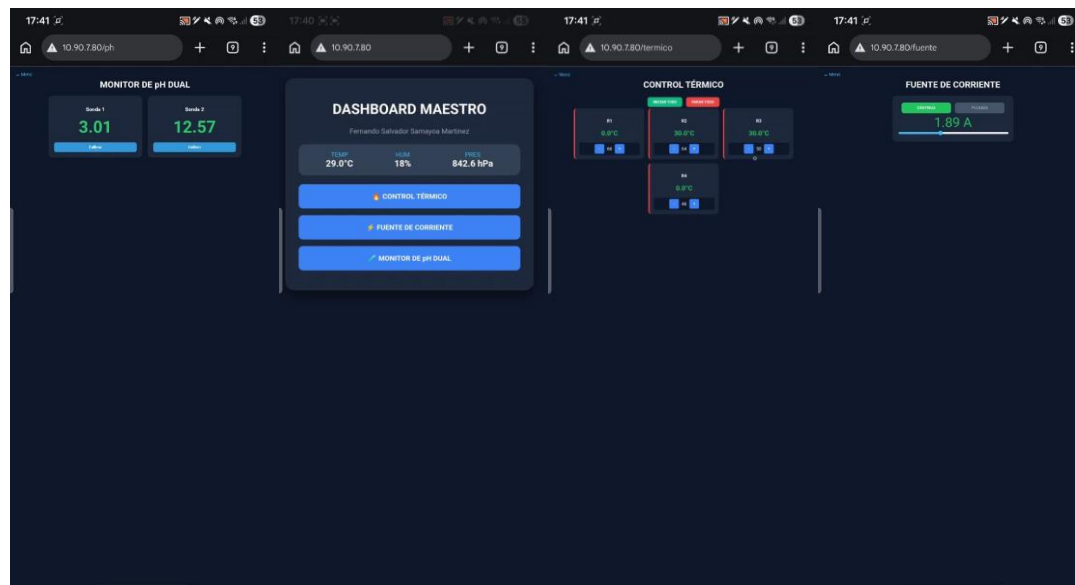


Imagen 7 Paneles de la interfaz

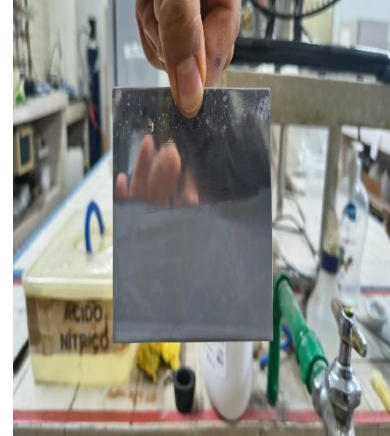
Una vez comprobado todo esto se procedió para la realización de las diversas etapas sobre las hojas de aluminio donde se pueden observar como las etapas fueron exitosas y un recubrimiento realizado sin control de temperatura, ph o corriente



*Imagen 8 Placa después de solución de limpieza con control a temperatura de 80 C*



*Imagen 9 Placa después de decapado a 85 C*



*Imagen 10 Placa zincada con 1.5 Amp CC a 30 C*



*Imagen 11 Placa Niquelada a Temperatura ambiente con 1.5 amp CC*



*Imagen 12 Placa de prueba sin control*

Como se puede observar con la validación física el sistema es funcional en todas las etapas por medio de la validación física tanto en interfaz como en actuadores además del cambio en los resultados finales con una placa donde se puede observar claramente el cambio de las regiones con diferente densidad de corriente y lo quemado en la otra

## 8. Conclusiones

Se logró el diseño y la implementación satisfactoria de un sistema de instrumentación y control robusto, fiable y especializado para procesos de recubrimiento electrolítico de Zinc-Níquel sobre sustratos de aluminio. El prototipo demostró una alta efectividad al reducir significativamente los tiempos muertos entre ensayos experimentales y optimizar la calidad de los acabados superficiales. Un factor diferenciador y de alto valor en ingeniería fue el desarrollo de la etapa de potencia con capacidad de corriente pulsada; esta característica proporciona ventajas cinéticas críticas en la electrodeposición que son difíciles de replicar con fuentes de alimentación comerciales de laboratorio dentro del rango de potencia requerido para este tipo de recubrimientos industriales. Como perspectivas de trabajos futuros y mejoras sustanciales al prototipo, se plantea la transición del cableado modular hacia el diseño e integración de los circuitos en una Tarjeta de Circuito Impreso (PCB) dedicada para minimizar el ruido electromagnético. Asimismo, se contempla optimizar la eficiencia energética del sumidero de corriente mediante topologías con menor disipación térmica, e integrar en el firmware algoritmos de gestión de energía como modos de reposo (sleep modes) y temporizadores precisos para las etapas inactivas. Finalmente, a nivel industrial, el sistema puede escalar hacia una línea de producción automatizada con ganchos transportadores mecánicos que minimicen la intervención humana, complementándose con un módulo de inspección de calidad automatizado basado en visión por computadora y algoritmos de inteligencia artificial para la clasificación y validación de los recubrimientos en tiempo real.

## 9. Bibliografía

- Ollard, E. A., & Smith, E. B. (1963). *Manual de recubrimientos electrolíticos industriales* (2.<sup>a</sup> ed.). Compañía Editorial Continental.
- Gutiérrez Pulido, H., & De la Vara Salazar, R. (2008). *Análisis y diseño de experimentos* (2.<sup>a</sup> ed.). McGraw-Hill.
- Casillas Santana, N., & Gudiño Guzman, B. (2024). *Electroquímica 2 para ingenieros químicos*. Universidad de Guadalajara.
- IEEE. (2020). *IEEE 802.11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications*.
- MathWorks. (2026). *Control System Toolbox User's Guide*
- Espressif Systems. (2026). *ESP32 Technical Reference Manual*
- Gutierrez Ramirez, V. U. (2026). *Desarrollo y validación de una formulación de zincado ácido para asegurar la adherencia en procesos de recubrimiento electrolítico sobre aluminio*. [Manuscrito no publicado] Universidad de Guadalajara.
- Ogata, K. (2010). *Ingeniería de control moderna* (5.<sup>a</sup> ed.). Pearson Educación.

**Rashid, M. H. (2015).** *Electrónica de potencia: Circuitos, dispositivos y aplicaciones* (4.<sup>a</sup> ed.). Pearson Educación.

## 10. Anexos

### Anexo 1 Código de sintonización PID

```
% =====  
% Código de sintonización PID  
% =====  
clear; clc; close all;  
  
%% 1. PANEL DE CONTROL (PARÁMETROS CONFIGURABLES)  
% -----  
T_amb      = 25.0;      % Temperatura ambiente inicial (°C)  
T_set      = 32.0;      % Temperatura objetivo / Setpoint (°C)  
P_max      = 18.0;      % Potencia máxima de la resistencia (Watts)  
L_retardo  = 10.0;      % Tiempo muerto térmico del sistema (Segundos)  
  
volumen_L  = .267;      % Volumen del líquido (Litros)  
A_s        = 0.029;     % Área superficial de disipación (m^2)  
h_efectivo = 65.0;      % Coeficiente de convección térmica (W/m^2*K)  
  
f_red      = 60;        % Frecuencia de la red eléctrica (Hz)  
% -----  
  
%% 2. MODELADO TERMODINÁMICO  
rho_crc = 1;            % Densidad del agua a 60°C (g/cm^3)  
m = volumen_L * rho_crc; % Masa real (kg)  
cp = 4184.3;           % Calor específico (J/kg*K)  
  
K = 1 / (h_efectivo * A_s);  
tau = (m * cp) / (h_efectivo * A_s);  
  
s = tf('s');  
G_planta = (K / (tau * s + 1)) * pade(exp(-L_retardo * s), 2);  
  
%% 3. SINTONIZACIÓN PID  
wc = 0.015;  
opciones = pidtuneOptions('DesignFocus', 'reference-tracking', 'PhaseMargin',  
65);  
C_pid = pidtune(G_planta, 'PIDF', wc, opciones);  
  
Kp = C_pid.Kp;  
Ki = C_pid.Ki;  
Kd = C_pid.Kd;  
  
if Kd <= 0  
    Kd = 0.0;  
end  
  
%% 4. SIMULACIÓN  
Ts = 1;  
t_sim = 2500; % Segundos de simulación
```

```

y_temp = ones(1, t_sim) * T_amb;
u_real = zeros(1, t_sim);
error = zeros(1, t_sim);

G_discreta = c2d(tf(K, [tau 1]), Ts);
[num, den] = tfdata(G_discreta, 'v');

integral = 0; error_prev = 0; buffer_retardo = zeros(1, L_retardo);

for k = 2:t_sim
    error(k) = T_set - y_temp(k-1);

    derivada = (error(k) - error_prev) / Ts;
    potencia_pedida = (Kp * error(k)) + (Ki * integral) + (Kd * derivada);

    % Saturación a P_max y Anti-Windup
    if potencia_pedida > P_max
        u_real(k) = P_max;
    elseif potencia_pedida < 0
        u_real(k) = 0;
    else
        u_real(k) = potencia_pedida;
        integral = integral + error(k) * Ts;
    end

    error_prev = error(k);

    % Dinámica de la planta con retardo
    u_aplicado = buffer_retardo(end);
    buffer_retardo = [u_real(k), buffer_retardo(1:end-1)];
    delta_y_actual = -den(2) * (y_temp(k-1) - T_amb) + num(2) * u_aplicado;
    y_temp(k) = delta_y_actual + T_amb;
end

tiempo_min = (1:t_sim) / 60;

%% 6. GRÁFICAS
% --- Ventana 1: Dinámica Temporal ---
figure('Name', sprintf('Dinámica del Sistema: %.2f m^2, %.1f W', A_s, P_max),
'Position', [50, 50, 800, 700]);

subplot(3,1,1);
plot(tiempo_min, y_temp, 'LineWidth', 2, 'Color', [0 0.447 0.741]); hold on;
yline(T_set, '--r', sprintf('Objetivo: %.1f °C', T_set),
'LabelHorizontalAlignment', 'left');
title(sprintf('Evolución de Temperatura (Volumen: %.1f L, Área: %.3f m^2)',
volumen_L, A_s));
ylabel('°C'); grid on; ylim([T_amb-2, max(T_set*1.1, max(y_temp)*1.05)]);

subplot(3,1,2);
plot(tiempo_min, u_real, 'LineWidth', 2, 'Color', [0.85 0.32 0.1]); hold on;
yline(P_max, '--k', sprintf('Límite: %.1f W', P_max),
'LabelHorizontalAlignment', 'left');

```

```

title(sprintf('Esfuerzo del Actuador (Saturado a %.1f W)', P_max));
ylabel('Watts'); grid on; ylim([-10, P_max*1.2]);

subplot(3,1,3);
plot(tiempo_min, error, 'LineWidth', 2, 'Color', [0.63 0.07 0.18]);
title('Error de Seguimiento'); xlabel('Tiempo (Minutos)'); ylabel('°C'); grid
on;

% --- Ventana 2: Estabilidad ---
figure('Name', 'Análisis de Estabilidad Térmica', 'Position', [870, 420, 600,
350]);
margin(C_pid * G_planta); grid on; title(sprintf('Diagrama de Bode (Planta %.1f
L)', volumen_L));

% --- Ventana 3: Actuador ---
figure('Name', 'Linealización TRIAC', 'Position', [870, 50, 600, 300]);
plot(P_norm * P_max, rad2deg(alpha_rad), 'LineWidth', 2, 'Color', [0.4660 0.6740
0.1880]);
title(sprintf('Perfil de Disparo del TRIAC (%.1f W Max)', P_max));
xlabel('Potencia Calculada (Watts)'); ylabel('Retardo \alpha (Grados)'); grid
on; axis tight;

```

Anexo 2 Código maestro ESP32-S3

```

#include <Wire.h>
#include <Adafruit_MCP4725.h>
#include <Adafruit_ADS1X15.h>
#include <Adafruit_AHTX0.h>
#include <Adafruit_BMP280.h>
#include <max6675.h>
#include <WiFi.h>
#include <WebServer.h>

/***** CONFIGURACIÓN RED *****/
const char* ssid = "SSID";
const char* password = "PASSWORD";
WebServer server(80);

/***** MÓDULOS AMBIENTALES *****/
Adafruit_AHTX0 aht;
Adafruit_BMP280 bmp;
float amb_temp = 0, amb_hum = 0, amb_pres = 0;

/***** MÓDULO pH (DUAL) *****/
Adafruit_ADS1115 ads;
float v7_1 = 2.5, v4_1 = 3.0, m_ph1 = -5.405, phActual1 = 7.0;
float v7_2 = 2.5, v4_2 = 3.0, m_ph2 = -5.405, phActual2 = 7.0;
bool calibradoPH1 = false, calibradoPH2 = false;

```

```

/***** MÓDULO FUENTE (DAC) *****/
Adafruit_MCP4725 dac;
bool modoPulsado = false;
int amplitudDAC = 1024, frecuencia = 1, dutyCycle = 50;
const float VREF = 5.0;

/***** MÓDULO TÉRMICO *****/
struct CanalTermico {
    int id; MAX6675* sensor;
    double temperatura, setpoint;
    bool activo;
    const double Kp, Ki, Kd;
    double integral, errorPrevio, limitePotencia;

    CanalTermico(int _id, int sck, int cs, int so, double p, double i, double
d)
        : Kp(p), Ki(i), Kd(d) {
        id = _id; sensor = new MAX6675(sck, cs, so);
        setpoint = 60; activo = false; integral = 0; errorPrevio = 0;
limitePotencia = 0;
    }

    double calcularPID() {
        if (!activo) return 0;
        double input = sensor->readCelsius();
        if (isnan(input)) return 0;
        temperatura = input;
        double error = setpoint - input;
        if (input >= setpoint) { integral = 0; return 0; }
        integral += error;
        double derivada = error - errorPrevio;
        errorPrevio = error;
        return constrain(Kp * error + Ki * integral + Kd * derivada, 0, 100);
    }
};

#define COMMON_SCK 18
#define COMMON_SO 19
CanalTermico canales[] = {
    CanalTermico(0, COMMON_SCK, 5, COMMON_SO, 62.65, 0.0897, 0.0),
    CanalTermico(1, COMMON_SCK, 4, COMMON_SO, 62.65, 0.0897, 0.0),
    CanalTermico(2, COMMON_SCK, 13, COMMON_SO, 16.71, 0.0239, 0.0),
    CanalTermico(3, COMMON_SCK, 14, COMMON_SO, 62.51, 0.0895, 0.0)
};

```

```

unsigned long lastPID = 0, lastRamp = 0, lastPH = 0, lastEnv = 0, lastIPMsg
= 0;

/***** FUNCIONES APOYO *****/
float leerVoltajePH(int canal) {
    long suma = 0;
    for (int i = 0; i < 15; i++) { suma += ads.readADC_SingleEnded(canal);
delay(2); }
    return (suma / 15.0) * 0.1875 / 1000.0;
}

/***** INTERFACES HTML *****/

void handleMenu() {
    String html = R"rawliteral(
<!DOCTYPE html><html><head><meta charset="UTF-8"><style>
body{font-family:sans-serif; background:#0f172a; color:white; text-
align:center; padding:20px;}
.menu-card{background:#1e293b; padding:30px; border-radius:20px;
display:inline-block; box-shadow:0 10px 30px rgba(0,0,0,0.5); width:90%;
max-width:500px;}
.env-box{display:flex; justify-content:space-around; background:#334155;
padding:15px; border-radius:15px; margin:20px 0;}
.env-item{font-size:0.9em; color:#38bdf8;} .env-val{font-size:1.4em;
display:block; color:white; font-weight:bold;}
.btn-menu{display:block; padding:18px; margin:15px auto; background:#3b82f6;
color:white; text-decoration:none; font-weight:bold; border-radius:10px;
transition:0.3s;}
.btn-menu:hover{background:#2563eb; transform:scale(1.02);}
</style></head><body>
<div class="menu-card">
    <h1>DASHBOARD MAESTRO</h1>
    <p style="color:#94a3b8">Fernando Salvador Samayoa Martinez</p>
    <div class="env-box">
        <div class="env-item">TEMP<span class="env-val" id="et">--</span></div>
        <div class="env-item">HUM<span class="env-val" id="eh">--</span></div>
        <div class="env-item">PRES<span class="env-val" id="ep">--</span></div>
    </div>
    <a href="/termico" class="btn-menu">💧 CONTROL TÉRMICO</a>
    <a href="/fuente" class="btn-menu">⚡ FUENTE DE CORRIENTE</a>
    <a href="/ph" class="btn-menu">🧪 MONITOR DE pH DUAL</a>
</div>
<script>
function updEnv(){
    fetch('/data_env').then(r=>r.json()).then(d=>{

```

```

        document.getElementById('et').innerText = d.t+"°C";
        document.getElementById('eh').innerText = d.h+"%";
        document.getElementById('ep').innerText = d.p+" hPa";
    });
}
setInterval(updEnv, 3000); updEnv();
</script></body></html>)rawliteral";
    server.send(200, "text/html", html);
}

void handleTermico() {
    String html = R"rawliteral(
<!DOCTYPE html><html><head><meta charset="UTF-8"><style>
body{font-family:sans-serif; background:#0f172a; color:white; text-align:center; padding:20px;}
.grid{display:flex; flex-wrap:wrap; justify-content:center; gap:15px;}
.card{background:#1e293b; border-radius:12px; padding:15px; width:220px; border-left:5px solid #64748b;}
.btn{padding:10px 20px; cursor:pointer; border-radius:8px; border:none; margin:5px; font-weight:bold;}
.nav-back{position:absolute; top:20px; left:20px; color:#38bdf8; text-decoration:none;}
.selector{display:flex; align-items:center; justify-content:center; gap:10px; margin-top:10px; background:#0f172a; padding:5px; border-radius:8px;}
.btn-step{background:#3b82f6; color:white; border:none; width:35px; height:35px; border-radius:5px; font-weight:bold; cursor:pointer;}
</style></head><body>
<a href="/" class="nav-back">← Menú</a>
<h1>CONTROL TÉRMICO</h1>
<button class="btn" style="background:#10b981; color:white;" onclick="control(1)">INICIAR TODO</button>
<button class="btn" style="background:#ef4444; color:white;" onclick="control(0)">PARAR TODO</button>
<div class="grid" id="cont"></div>
<script>
function update(){
    fetch('/data_t').then(res=>res.json()).then(data=>{
        let c = document.getElementById('cont'); c.innerHTML = "";
        data.forEach((d,i)=>{
            c.innerHTML += `<div class="card" style="border-left-color:${d.run?'#22c55e':'#ef4444'}">
                <h4>R${i+1}</h4><h2 style="color:#22c55e">${d.t.toFixed(1)}°C</h2>
                <div class="selector">
                    <button class="btn-step" onclick="adj(${i},-1)">-</button>

```

```

        <span id="sp_${i}">${d.sp}</span>
        <button class="btn-step" onclick="adj(${i},1)">+</button>
    </div></div>`;
    });
});
}
function adj(id, step){
    let el = document.getElementById('sp_'+id);
    let newVal = parseInt(el.innerText) + step;
    if(newVal<0) newVal=0; if(newVal>150) newVal=150;
    el.innerText = newVal;
    fetch(`/set_t?id=${id}&v=${newVal}`);
}
function control(s){ fetch(`/act_t?run=${s}`); }
setInterval(update,1500);
</script></body></html>)rawliteral";
    server.send(200, "text/html", html);
}

void handleFuente() {
    float amperios = ((amplitudDAC * VREF) / 4095.0) * 2.0;
    String html = R"rawliteral(
<!DOCTYPE html><html><head><meta charset="UTF-8"><style>
body{font-family:sans-serif; background:#0f172a; color:white; text-
align:center; padding:20px;}
.card{background:#1e293b; border-radius:15px; padding:20px; margin:10px
auto; max-width:400px;}
.amp{font-size:48px; color:#22c55e;}
.btn{padding:12px 20px; cursor:pointer; border-radius:10px; border:none;
margin:5px; font-weight:bold; width:45%;}
.active{background:#22c55e; color:white;}
.inactive{background:#475569; color:#94a3b8;}
.nav-back{position:absolute; top:20px; left:20px; color:#38bdf8; text-
decoration:none;}
input[type=range]{width:100%; accent-color:#38bdf8;}
.hidden{display:none;}
</style></head><body>
<a href="/" class="nav-back">← Menú</a>
<h1>FUENTE DE CORRIENTE</h1>
<div class="card">
    <button id="btnC" class="btn )rawliteral" + String(!modoPulsado ? "active"
: "inactive") + R"rawliteral(" onclick="setM(0)">CONTINUA</button>
    <button id="btnP" class="btn )rawliteral" + String(modoPulsado ? "active"
: "inactive") + R"rawliteral(" onclick="setM(1)">PULSADA</button>

```

```

    <div class="amp"><span id="at">)rawliteral" + String(amperios, 2) +
R"rawliteral(</span> A</div>
    <input type="range" min="0" max="2047" value=")rawliteral" +
String(amplitudDAC) + R"rawliteral(" oninput="upd('a',this.value)">
    <div id="cont-pulsada" class=")rawliteral" + String(modoS Pulsado ? "" :
"hidden") + R"rawliteral(">
        <p>Frecuencia: <span id="fv">)rawliteral" + String(frecuencia) +
R"rawliteral(</span> Hz</p>
        <input type="range" min="1" max="100" value=")rawliteral" +
String(frecuencia) + R"rawliteral(" oninput="upd('f',this.value)">
        <p>Duty Cycle: <span id="dv">)rawliteral" + String(dutyCycle) +
R"rawliteral(</span> %</p>
        <input type="range" min="1" max="99" value=")rawliteral" +
String(dutyCycle) + R"rawliteral(" oninput="upd('d',this.value)">
    </div>
</div>
<script>
function setM(m){
    fetch("/modo_f?v="+m).then(() => {
        document.getElementById('btnC').className = (m==0)?'btn active':'btn
inactive';
        document.getElementById('btnP').className = (m==1)?'btn active':'btn
inactive';
        document.getElementById('cont-pulsada').className = (m==1)?'':'hidden';
    });
}
function upd(p,v){
    if(p=='a') document.getElementById('at').innerText =
((v*5/4095)*2).toFixed(2);
    else if(p=='f') document.getElementById('fv').innerText = v;
    else if(p=='d') document.getElementById('dv').innerText = v;
    fetch("/set_f?p="+p+"&v="+v);
}
</script></body></html>)rawliteral";
    server.send(200, "text/html", html);
}

void handlePH() {
    String html = R"rawliteral(
<!DOCTYPE html><html><head><meta charset="UTF-8"><style>
body{font-family:sans-serif; background:#0f172a; color:white; text-
align:center; padding:20px;}
.grid{display:flex; flex-wrap:wrap; justify-content:center; gap:20px;}
.card{background:#1e293b; border-radius:15px; padding:20px; width:300px;}
.ph-val{font-size:60px; color:#2ecc71; font-weight:bold; margin:10px 0;}

```

```

.btn{background:#3498db; color:white; border:none; padding:10px; border-
radius:8px; cursor:pointer; width:100%; font-weight:bold; margin-top:10px;}
.nav-back{position:absolute; top:20px; left:20px; color:#38bdf8; text-
decoration:none;}
.step-box{display:none; background:#334155; padding:15px; border-
radius:10px; margin-top:15px;}
</style></head><body>
<a href="/" class="nav-back">← Menú</a>
<h1>MONITOR DE pH DUAL</h1>
<div class="grid">
  <div class="card">
    <h3>Sonda 1</h3>
    <div id="ph1" class="ph-val">--</div>
    <div id="main1"><button class="btn"
onclick="startCal(1)">Calibrar</button></div>
    <div id="cal1" class="step-box"><p id="instr1">pH 7</p><button id="btn1"
class="btn" onclick="nextStep(1)">LISTO</button></div>
  </div>
  <div class="card">
    <h3>Sonda 2</h3>
    <div id="ph2" class="ph-val">--</div>
    <div id="main2"><button class="btn"
onclick="startCal(2)">Calibrar</button></div>
    <div id="cal2" class="step-box"><p id="instr2">pH 7</p><button id="btn2"
class="btn" onclick="nextStep(2)">LISTO</button></div>
  </div>
</div>
<script>
let step1=1, step2=1;
setInterval(()=>{ if(step1==1 && step2==1)
fetch('/get_ph_dual').then(r=>r.json()).then(d=>{document.getElementById('ph
1').innerText=d.p1;document.getElementById('ph2').innerText=d.p2;});},
1500);
function startCal(id){
if(id==1){step1=2;document.getElementById('main1').style.display='none';docu
ment.getElementById('cal1').style.display='block';}else{step2=2;document.get
ElementById('main2').style.display='none';document.getElementById('cal2').st
yle.display='block';}}
function nextStep(id){
  let s=(id==1)?step1:step2; let p=(s==2)?7:4;
  fetch(`/do_cal_ph?id=${id}&p=${p}`).then(()=>{
    if(p==7){ if(id==1)step1=3;else step2=3;
document.getElementById('instr'+id).innerHTML='pH 4'; }
    else { alert('Sonda '+id+' OK'); location.reload(); }
  });
};

```

```

}
</script></body></html>>rawliteral";
    server.send(200, "text/html", html);
}

/***** SETUP & LOOP *****/
void setup() {
    Serial.begin(115200);
    delay(1000);
    Wire.begin(8, 9);

    Serial2.begin(9600, SERIAL_8N1, -1, 17);

    dac.begin(0x60);
    ads.begin();
    aht.begin();
    bmp.begin(0x77);

    WiFi.begin(ssid, password);
    while(WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
    Serial.println("\nIP: " + WiFi.localIP().toString());

    server.on("/", handleMenu);
    server.on("/termico", handleTermico);
    server.on("/fuente", handleFuente);
    server.on("/ph", handlePH);

    server.on("/data_env", [](){
        String j =
        "{\n\t\"t\": \""+String(amb_temp,1)+"\", \"h\": \""+String(amb_hum,0)+"\", \"p\": \""
+String(amb_pres,1)+"\"}";
        server.send(200, "application/json", j);
    });

    server.on("/get_ph_dual", [](){
        String j =
        "{\n\"p1\": \""+String(phActual1,2)+"\", \"p2\": \""+String(phActual2,2)+"\"}";
        server.send(200, "application/json", j);
    });

    server.on("/data_t", [](){
        String j = "[";
        for(int i=0; i<4; i++) j +=
        "{\n\t\"t\": "+String(canales[i].temperatura)+"\", \"sp\": "+String(canales[i].setpoi
nt)+"\", \"run\": "+String(canales[i].activo)+ "}" + (i<3?"", ":");

```

```

        server.send(200, "application/json", j + "]);
    });

    server.on("/set_t", [](){
        int id=server.arg("id").toInt();
        if(id>=0 && id<4) canales[id].setpoint = server.arg("v").toDouble();
        server.send(200, "text/plain", "OK");
    });

    server.on("/act_t", [](){
        bool s=server.arg("run").toInt();
        for(int i=0;i<4;i++){ canales[i].activo=s; if(!s){
canales[i].limitePotencia=0; canales[i].integral=0; } }
        server.send(200, "text/plain", "OK");
    });

    server.on("/set_f", [](){
        char p=server.arg("p")[0]; int v=server.arg("v").toInt();
        if(p=='a') amplitudDAC=v; else if(p=='f') frecuencia=v; else if(p=='d')
dutyCycle=v;
        server.send(200, "text/plain", "OK");
    });

    server.on("/modo_f", [](){ modoPulsado=(server.arg("v")=="1");
server.send(200, "text/plain", "OK"); });

    server.on("/do_cal_ph", [](){
        int id = server.arg("id").toInt(); int p = server.arg("p").toInt();
        float v = leerVoltajePH(id-1);
        if(id==1){ if(p==7) v7_1=v; if(p==4){ v4_1=v; m_ph1=(4.0-7.0)/(v4_1-
v7_1); calibradoPH1=true; } }
        else { if(p==7) v7_2=v; if(p==4){ v4_2=v; m_ph2=(4.0-7.0)/(v4_2-v7_2);
calibradoPH2=true; } }
        server.send(200, "text/plain", "OK");
    });

    server.begin();
}

void loop() {
    server.handleClient();

    // Lectura Ambiental cada 5 seg
    if (millis() - lastEnv >= 5000) {
        lastEnv = millis();
    }
}

```

```

    sensors_event_t h_event, t_event;
    aht.getEvent(&h_event, &t_event);
    amb_temp = t_event.temperature;
    amb_hum = h_event.relative_humidity;
    amb_pres = bmp.readPressure() / 100.0F;
}

// Lectura pH cada 2 seg
if (millis() - lastPH >= 2000) {
    lastPH = millis();
    float v1 = leerVoltajePH(0), v2 = leerVoltajePH(1);
    phActual1 = calibradoPH1 ? (7.0 + (v7_1 - v1) * m_ph1) : (7.0 + (2.5 -
v1) / 0.185);
    phActual2 = calibradoPH2 ? (7.0 + (v7_2 - v2) * m_ph2) : (7.0 + (2.5 -
v2) / 0.185);
}

// PID y Rampa Térmica
if (millis() - lastRamp >= 200) {
    lastRamp = millis();
    for(int i=0; i<4; i++) if(canales[i].activo && canales[i].limitePotencia
< 100) canales[i].limitePotencia++;
}

if (millis() - lastPID >= 1000) {
    lastPID = millis();
    String s = "";
    for(int i=0; i<4; i++) {
        int val = (int)min(canales[i].calcularPID(),
canales[i].limitePotencia);
        s += String(val);
        if(i<3) s += ",";
    }
    Serial2.println(s); // Envía al Nano a 9600 baudios
}

// Salida DAC
int vOut = amplitudDAC;
if (modoPulsado) {
    unsigned long p = 1000000 / frecuencia;
    vOut = ((micros() % p) < (p * dutyCycle / 100)) ? amplitudDAC : 0;
}
dac.setVoltage(vOut, false);
}

```

### Anexo 3 Código Slave Arduino NANO

```
#include <AcPhaseControl.h>

/***** CONFIGURACIÓN DE PINES *****/
int ZC_PIN = 3;
int GATE_PINS[] = {7, 8, 9, 10};
int NUM_CANALES = 4;

AcPhaseControl dimmer;

/***** VARIABLES DE CONTROL *****/
unsigned long windowSize = 3000;
unsigned long windowStart = 0;
int potencias[4] = {0, 0, 0, 0};

void setup() {
    Serial.begin(9600);

    for(int i = 0; i < NUM_CANALES; i++) {
        dimmer.begin(GATE_PINS[i], ZC_PIN);
    }

    windowStart = millis();
}

void loop() {
    // 1. RECEPCIÓN DE DATOS
    if (Serial.available() > 0) {
        // Leemos los 4 valores enviados como: val1,val2,val3,val4\n
        potencias[0] = Serial.parseInt();
        potencias[1] = Serial.parseInt();
        potencias[2] = Serial.parseInt();
        potencias[3] = Serial.parseInt();

        // Limpiamos cualquier residuo en el buffer (\n o espacios)
        while(Serial.available() > 0) { Serial.read(); }

        // Aseguramos que los valores sean válidos
        for(int i=0; i<4; i++) {
            potencias[i] = constrain(potencias[i], 0, 100);
        }
    }

    // 2. LÓGICA DE TIEMPO
```

```
unsigned long now = millis();
if (now - windowStart >= windowSize) {
    windowStart = now;
}

// 3. CONTROL DE SALIDAS
for (int i = 0; i < NUM_CANALES; i++) {
    unsigned long tiempoEncendido = (windowSize * (unsigned
long)potencias[i]) / 100;

    if (potencias[i] == 0) {
        dimmer.APC_OFF(GATE_PINS[i]);
    }
    else if ((now - windowStart) < tiempoEncendido) {
        dimmer.APC_ON(GATE_PINS[i]);
    }
    else {
        dimmer.APC_OFF(GATE_PINS[i]);
    }
}
}
```